Feedback-Controlled Adaptive Level of Detail for Vegetation Rendering in the Godot Engine

Shreeniwas Salian
Department of Computer Engineering
Pimpri Chinchwad College of Engineering

Koustubh Tayade
Department of Computer Engineering
Pimpri Chinchwad College of Engineering

Guide: Pravin Game
Department of Computer Engineering, Pimpri Chinchwad College of Engineering

Abstract*—We present a feedback-controlled level-of-detail (LOD) system for dense vegetation in the Godot 4 engine and evaluate it against two references: no LOD (every instance at full detail) and a fixed-threshold distance LOD. The adaptive system keeps the baseline's four distance policies (trees, grass, bushes, debris) and scales their thresholds with one bias that a proportional-integral (PI) controller derives from measured GPU render time, bounded by distance floors, per-category caps, rate limits and a per-update change budget. On a Terrain3D scene with 41,698 baked instances and 403,088 procedural grass clumps, with five runs per cell on a Quadro P3200 laptop GPU (debug build), distance LOD raises average FPS from 21.4 to 96.7 at 1080p and from 19.6 to 70.5 at 1440p and removes about 90% of rendered primitives. The PI controller adds little on top of the fixed baseline: +0.2% FPS at 1080p, where the GPU stays under its 16.67 ms target and the bias remains 0, and +2.4% at 1440p, where the bias saturates in 84% of frames and the 8 ms target is still not reached. It also cuts policy CPU time from 3.3 to 0.34 ms per frame through update scheduling. We report the method, the benchmark protocol, the measurements and the limits of the evidence.

***Index Terms*—level of detail, adaptive rendering, PI control, vegetation rendering, MultiMesh, Godot engine, real-time performance**

I. Introduction

Dense vegetation is one of the costliest parts of an outdoor real-time scene: thousands of trees and hundreds of thousands of grass clumps multiply triangle counts, draw calls and overdraw. Level of detail (LOD) reduces this cost by drawing simpler representations of distant objects [1]–[4]. The simpler representations come from mesh simplification [5]–[7]; vegetation is usually handled with instancing, billboards and density thinning [8]–[12]; terrain has its own continuous LOD [13]; and recent engines virtualize geometry [14]. Most engines, including Godot, expose LOD as fixed distances chosen by an artist. Fixed distances cannot react to the frame budget: on a fast GPU they discard detail that would have been affordable, and on a slow GPU they keep detail that makes the frame miss its target.

Closed-loop control offers an alternative. Instead of fixing the distances, a controller measures how long the GPU takes and moves one global quality knob up or down [15], [16]. Feedback LOD is an old idea [17] and PI/PID control is standard engineering practice [18], but open, reproducible measurements of it on vegetation in a freely available engine are scarce, and it is rarely separated from the gain that plain distance LOD already provides.

This paper makes four contributions. (1) A multi-category vegetation LOD for Godot 4 (trees with three mesh levels, grass with three tiers plus cull, bushes and debris with density thinning) built on chunked MultiMesh data, with no per-instance nodes and no change to collision or saved scenes. (2) An adaptive mode in which one PI-controlled bias scales all thresholds in metres, bounded by floors that protect near-field detail, per-category caps, rate limits, a dead band and a per-update change budget. (3) A reproducible benchmark with one scene, one seeded 528 m camera path, interleaved runs, per-frame CSV and JSON logs, and three modes (None, Baseline, Adaptive) measured under identical conditions. (4) A measured, three-way decomposition of the gain: how much comes from having distance LOD at all (None to Baseline) and how much the controller adds (Baseline to Adaptive).

The rest of the paper is organized as follows. Section II reviews related work, Section III describes the system, Section IV the experimental method, Section V the results, and Sections VI and VII discuss limitations and conclude.

II. Related Work

Feedback-driven LOD predates modern GPUs. Funkhouser and Séquin [15] choose object detail each frame to keep a target frame time, Luebke et al. [3] survey the field, and Wimmer and Wonka [16] estimate rendering time so that a frame budget can be met. Wong and Wang [17] apply PID control to frame-rate control in interactive rendering, motivated by the observation that plain LOD does not bound frame rate. Our controller follows this line, using a standard PI design [18], but acts on distance thresholds of instanced vegetation and is evaluated against a no-LOD reference as well as a fixed-threshold reference.

Table I lists recent work (2020 onward) that is closest to ours. Reported figures are those stated by the authors; hardware, scenes and baselines differ, so they give context rather than a like-for-like ranking. The literature search behind this table was limited and not systematic. Reference [19] uses frame-budget-driven adaptive LOD with progressive meshes; [20] generates trees on the GPU with work graphs and continuous LOD; [21] uses clustered LOD with opacity micromaps for ray-traced foliage; [22] adapts LOD updates using CPU usage and spatial hashing; and [23] refines terrain adaptively on the GPU.

TABLE I
Recent Related Work and Reported Results

Work	Reported result	Setup
Adaptive LOD, VR interiors, 2026 [19]	High load: p95 GPU frame time 18.11 to 10.86 ms; frames over budget 84.51% to 7.01% vs full resolution	VR scenes, 11.11 ms budget
GPU tree generation, HPG 2025 [20]	Median frame 7.74 ms for the whole scene; 51 KiB instead of 34.8 GiB of meshes	RX 7900 XTX, 1,200 trees
Foliage path tracing, ACM [21]	About 60 FPS	RTX 4090, 1440p to 2160p

Work	Reported result	Setup
CPU-usage LOD, 2024 [22]	CPU time 15.1 to 14.2 ms (−5.96%) and 10.8 to 10.3 ms (−4.63%); triangles −15.79% and −25%; FPS 66.1 to 90 and 121 to 146	Two areas; hardware not stated
Concurrent binary trees, 2020 [23]	Linear speed-up with processor count; timings not extracted	GPU terrain

Two observations follow. First, the closest work in intent [19] reports its gain against full-resolution rendering, as we do against None; our reductions (Section V) are larger in magnitude, but our scene, hardware and baseline are different and most of our gain comes from distance LOD itself rather than from the controller. Second, in our limited search we found no published result that isolates the effect of a feedback controller from that of the underlying distance LOD for vegetation, which is what our three-mode protocol is designed to do.

III. System Design

The system is written in GDScript for Godot 4.7.2 [24] on top of Terrain3D 1.0.2 [25]; Terrain3D's own LOD is never modified. A single node, LodManager, switches between three modes: None (all policies disabled, baked vegetation restored), Baseline (the four distance policies with fixed thresholds) and Adaptive (the same policies with thresholds scaled by one controlled bias). Fig. 1 shows the adaptive pipeline.

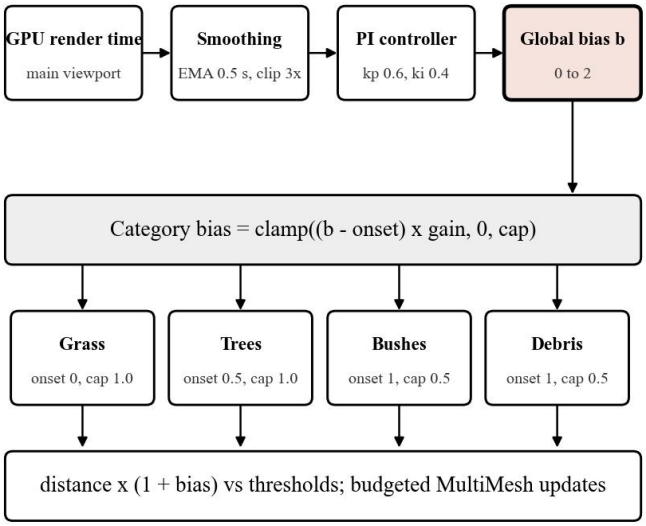


Fig. 1. Adaptive LOD pipeline. GPU render time is smoothed and fed to a PI controller whose output, the global bias, is mapped to a per-category bias that scales each policy's distance thresholds.

A. Vegetation Categories and Baseline Policies

All policies use chunked MultiMesh data, never create per-instance nodes, and never touch collision or the saved scenes (Table II). Bushes and debris have a single mesh, so they are thinned rather than simplified. Density tiers are nested prefixes of a fixed pseudo-random order, so an instance never blinks between tiers; it only appears or disappears.

TABLE II
Vegetation Categories and Baseline LOD

Category	Instances	Baseline LOD	Hyst.
Trees	9,816	Three mesh levels per instance: LOD0 under 60 m, LOD1 to 150 m, LOD2 beyond	10 m
Grass	403,088 clumps	32 m cells, tiers at 25/60 m, cull at 110 m; densities 100/40/12%	6 m
Bushes	7,615	Density per 64 m chunk: 100, 60, 30, 10, 0% at 50/100/160/240 m	8 m
Debris	1,341	Density: 100, 50, 20, 0% at 30/60/110 m	5 m
Rocks	606	Metrics only, no LOD	n/a

B. Bias and Thresholds

The adaptive mode introduces a bias b, positive meaning coarser, with $b = 0$ reproducing the Baseline exactly. A policy compares the camera distance multiplied by $(1 + b)$ against its unchanged thresholds, so $b = 1$ halves every threshold in metres. For trees the effective threshold is clamped by a floor and never exceeds its baseline value:

$$T_i^{\text{eff}} = \min(\, T_i, \max(\, F_i, \, T_i \,/\, (1 + b_{\text{trees}})\,)\,) \tag{1}$$

where T_i are the baseline thresholds (60 and 150 m) and the floors are $F_1 = 60$ m and $F_2 = 100$ m. The floors mean that trees closer than 100 m keep their baseline mesh level under any bias, so adaptive pressure can only move the 100 to 150 m band of LOD1 trees to LOD2.

C. PI Controller

The controlled signal is the GPU render time of the main viewport, read from the engine's measured render time, which excludes VSync waits and frame caps. If it is unavailable and VSync or a frame cap is on, the bias is held. The signal is smoothed by an exponential moving average and clipped so that one sample cannot count for more than three times the target. With target t, smoothed signal s and relative error $e = (s - t) / t$, the controller computes

$$e_{eff} = \text{deadband}(e, 0.03), \;\; P = k_p e_{eff}, \;\; I \leftarrow I + k_i e_{eff} \Delta t \tag{2}$$

$$b^{*} = \text{clamp}(\, P + I, \, b_{\min}, \, b_{\max}\,) \tag{3}$$

$$b = \text{clamp}(\, b^{*}, \, b_{prev} - r_{\text{fall}}, \, b_{prev} + r_{\text{rise}}\,) \tag{4}$$

The integral term is frozen while the output is saturated in the direction of the error and is back-calculated when the rate limit binds, which prevents wind-up [26], [27]. Feedback control of computing systems is well established [28]; classical tuning rules such as Ziegler–Nichols [29] were not applied, and the gains in Table III are conservative first choices that were not tuned.

TABLE III
Controller Parameters

Parameter	Value	Meaning
Target	16.67 ms (1080p), 8 ms (1440p)	GPU time to stay under
k_p, k_i	0.6, 0.4	Proportional and integral gains
$b_{\min}$, $b_{\max}$	0, 2	Never finer than baseline; thresholds divided by up to 3
Rate limits	+0.10 rise, −0.03 fall per 0.25 s	Coarsen fast, recover about 3x slower
Dead band	3% of target	No reaction inside it
Smoothing	EMA, 0.5 s	Signal filter

D. Priority, Caps and Smoothness

The global bias is distributed in priority order so that the cheapest-to-lose detail goes first:

$$b_c = \text{clamp}(\ (\ b - \text{onset}_c\ ) \times \text{gain}_c,\ b_{\min},\ \text{cap}_c\ ) \qquad (5)$$

Grass starts at onset 0 (cap 1.0), trees at 0.5 (cap 1.0), and bushes and debris at 1.0 with gain 0.5 (cap 0.5), because hiding bushes and debris measurably changed GPU time by only 0.0 to 0.1 ms. Policy updates run every 0.1 s, staggered by a quarter interval so no two categories update on the same frame, skip updates when the camera has moved less than 0.5 m and nothing is pending, and are limited to a change budget per update (200 tree instances, 30 grass cells, 20 bush or debris chunks) with a rotating cursor. A budget-limited run converges to the same result as an unlimited one. A tree still switches mesh in a single frame: there is no cross-fade, so mesh pops are limited in distance, rate and burst size but not removed.

IV. Experimental Methodology

All three modes run in the same scene, with the same seed (1337) and the same 528 m camera path at 8 m/s. Each run has a 10 s warm-up, in which policy data are built, followed by 60 s of measurement; the controller is reset to bias 0 at the start of measurement. Two profiles are used: Normal (1920×1080, controller target 16.67 ms) and Stress (2560×1440, target 8 ms, an experimental target rather than a promise). VSync is off and FPS is uncapped. Each mode and resolution was run five times, 30 runs in total, with the order rotated between repetitions; none was discarded. Table IV gives the test configuration. The Baseline stands for the artist-set fixed distances typical of game engines [30]. Following common practice in performance measurement [31]–[33], we report means, standard deviations and per-run pairs rather than single runs.

TABLE IV
Test Configuration

Item	Value
Engine	Godot 4.7.2-stable, Forward+ on Direct3D 12, no MSAA or TAA
Build	Debug and editor build (no export templates installed, so no release build)
GPU, driver	NVIDIA Quadro P3200 (laptop), driver 582.16
CPU, memory	Intel i7-8850H, 64 GB
Scene	Terrain3D 1.0.2; 41,698 baked instances plus 403,088 procedural grass clumps
Metrics	Per-frame frame time and GPU time, draw calls, rendered primitives, LOD switches, policy CPU time, bias

Average FPS is the mean over frames. The 1% low is 1000 divided by the 99th-percentile frame time. GPU time is the engine's measured render time for the main viewport and is distinct from CPU or process time. Changes are computed as (Adaptive − reference) / reference, and Baseline and Adaptive are compared as pairs by repetition slot. Correctness was checked by four self-test suites run through an external runner that fails a run on any script error or unexpected engine message (124, 90, 51 and 17 checks passed; the grass determinism stage was skipped).

V. Results

A. Three-Mode Comparison

Distance LOD accounts for almost all of the gain: Baseline runs 4.5 times (1080p) and 3.6 times (1440p) faster than None, and Adaptive adds +0.2% and +2.4% over Baseline (Table V, Fig. 2).

TABLE V
Mean of Five 60 s Runs per Cell (Debug Build)

Res.	Mode	FPS ± sd	1% low	Frame ms	p99 ms	GPU ms
1080p	None	21.4 ± 0.1	13.9	46.72	72.0	46.27
1080p	Baseline	96.7 ± 0.3	77.2	10.35	13.0	9.86
1080p	Adaptive	96.8 ± 0.3	77.6	10.33	12.9	9.87
1440p	None	19.6 ± 0.0	13.0	51.15	77.0	50.57
1440p	Baseline	70.5 ± 0.2	55.7	14.19	18.0	13.59
1440p	Adaptive	72.1 ± 0.1	57.3	13.87	17.4	13.29

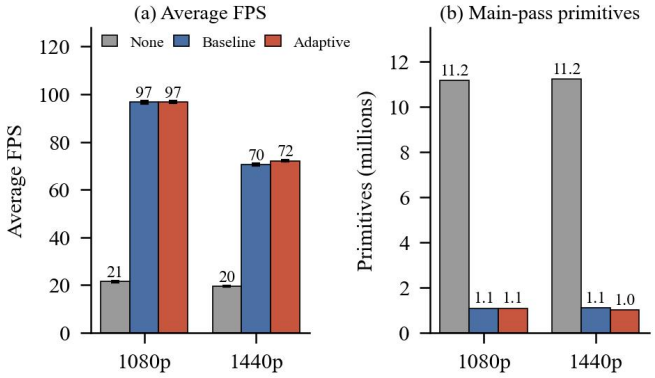


Fig. 2. Three-mode comparison at 1080p and 1440p. (a) Average FPS (error bars: standard deviation over five runs). (b) Mean main-pass primitives per frame.

Against None, Adaptive reduces mean frame time by 77.9% at 1080p and 72.9% at 1440p, and p99 frame time from 72.0 to 12.9 ms and from 77.0 to 17.4 ms. No frame fell below 10 FPS in any run; the lowest per-run FPS in the None and Adaptive runs was 12.7 and 44.7 respectively.

B. Why: Geometry

TABLE VI
Mean Per-Frame Counters

Res.	Mode	Primitives	Draw calls	Policy CPU ms	Switches
1080p	None	11.15 M	3,120	n/a	0
1080p	Baseline	1.08 M	592	3.3	4,077
1080p	Adaptive	1.08 M	591	0.34	4,076
1440p	None	11.22 M	3,137	n/a	0
1440p	Baseline	1.09 M	594	3.5	4,077
1440p	Adaptive	1.00 M	552	0.42	3,277

Distance LOD cuts rendered primitives by about 90% and main-pass draw calls by about 81%. Adaptive removes a further 8% of primitives and 7% of draw calls at 1440p, but only about 2% of GPU time (Fig. 3): fewer primitives do not translate proportionally into GPU time.

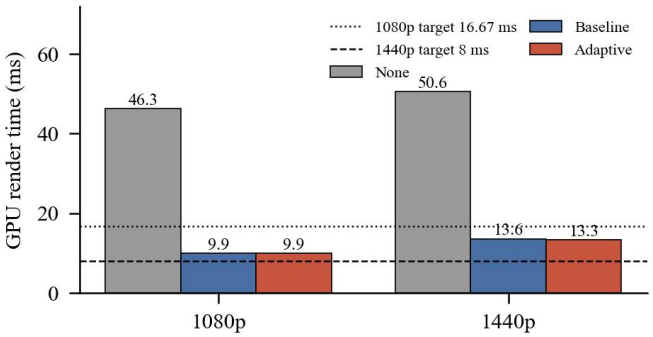


Fig. 3. Mean GPU render time per mode. The dotted and dashed lines mark the controller targets for 1080p and 1440p.

C. What the Controller Adds

At 1080p the GPU time (9.9 ms) is already under the 16.67 ms target, the bias mean is 0.00 and it never reaches its maximum, so Adaptive equals Baseline (paired differences +0.1, +0.4, +0.2, +0.2 and −0.2%). At 1440p the GPU time (13.6 ms) is above the 8 ms target, the bias mean is 1.86 and it is at its maximum in 84% of frames; Adaptive is faster in all five pairs (+2.9, +2.6, +1.8, +2.6 and +2.0%), but the 8 ms target is not reached even at maximum bias (Fig. 4). In a supplementary 1080p run with the target lowered to 8 ms (three runs per mode), Baseline averaged 97.0 FPS and Adaptive 99.5 FPS (+2.6%, bias at maximum in 60% of frames), so the controller helps when, and only when, the frame is over its target.

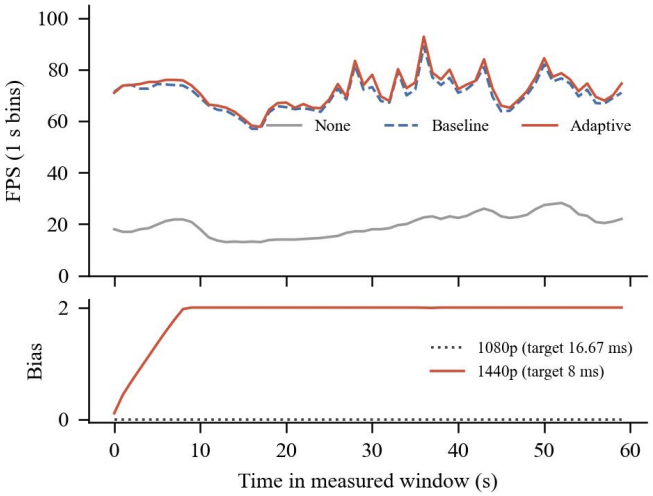


Fig. 4. Top: FPS over time at 1440p, mean of five runs per mode. Bottom: adaptive bias over time at both resolutions. At 1080p the bias stays at 0.

The controller also lowers its own cost. Through update scheduling, the CPU time of the four policies fell from 3.3 to 0.34 ms per frame at 1080p and from 3.5 to 0.42 ms at 1440p (debug-build GDScript; the runs are GPU-bound, so this does not appear in FPS).

D. Comparison with CPU-Driven Adaptive LOD

The CPU-usage-driven LOD of [22] is the closest in aim. Its gains resemble our Baseline-to-Adaptive step more than our None-to-Adaptive step, because it adapts an existing LOD rather than adding one, although its reference configuration is not stated here. Measured that way, its FPS gains (+36.2% and +20.7%) and triangle reductions (15.79% and 25%) are larger than ours (+0.2% and +2.4% FPS; 0.2% and 8% fewer primitives at 1080p and 1440p). The difference fits the workloads: its signal is CPU usage in what appear to be CPU-bound areas, while ours is GPU render time in a GPU-bound scene, where fewer primitives buy little GPU time and the controller is idle at 1080p. The two signals are complementary, and the two systems were not measured on the same scene or hardware.

E. Visual Comparison

With the nearby-tree floors in place, adaptive coarsening is confined to mid-field grass thinning, a little distant tree simplification and thinner bushes (Fig. 5). In fixed-view probes at three path positions (three repetitions, bias pinned at 1.0, 1.5 and 2.0), nearby trees were identical to Baseline and main-pass primitives fell by 1 to 10% relative to Baseline, with a mean absolute pixel difference from the Baseline frame of 0.7 to 4.3 on a 0 to 255 scale. These differences include grass wind animation and are not a pure LOD measure; no perceptual metric (SSIM [34], FLIP [35] or HDR-VDP-2 [36]) was computed. Fig. 6 shows the in-game debug panel, which exposes only None and Adaptive.



Fig. 5. Same viewpoint, 2x crop. Left: None (full detail). Right: Adaptive under a running controller.

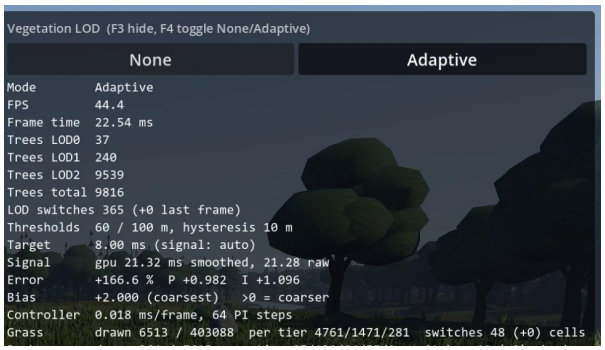


Fig. 6. In-game debug panel (F3) in Adaptive mode: per-category counts, controller target, signal, error and bias.

VI. Discussion and Limitations

The central result is a decomposition. Most of the benefit that could be credited to “adaptive” LOD in a two-way comparison with no LOD comes from distance LOD itself; the feedback controller is a bounded safety margin that engages when the GPU is over its target. A None-versus-Adaptive comparison alone would attribute a 4.5-fold speed-up to the controller, while the three-mode comparison attributes about 0.2 to 2.4% of it. The controller behaves as designed: it coarsens quickly, clamps at its limits, stays stable, and holds the bias at 0 when there is nothing to fix. Its benefit is limited by the workload, since terrain, shadows and the rest of the frame dominate GPU time and vegetation LOD alone cannot reach the 8 ms target at 1440p.

The limitations are as follows. (1) All timings come from a debug and editor build on one laptop GPU; no release build was possible, so absolute numbers should not be compared with release results. (2) The GPU drifted by a factor of two to three between earlier sessions. The series reported here stayed stable (standard deviation of average FPS 0.03 to 0.32), but an earlier 1080p gain of +12.5% (three runs, GPU in a slower state) did not reproduce and is not claimed. (3) Five runs per cell give a consistent sign but no claim of general reliability, and controller gains and thresholds are untuned. (4) Importance is category-level only; there is no per-instance or screen-coverage importance. (5) Tree mesh switches still pop because there is no cross-fade, and no perceptual image metric was computed. (6) The GPU signal lags by two to three frames and cannot see CPU-bound frames. (7) The grass determinism test stage was not run in the final test series.

Other ways to reduce rendering cost exist and could be combined with the controller: perceptually modulated LOD [37], foveated rendering [38], [39] and variable-rate or coarse-pixel shading [40]. They need extra input (gaze, saliency or shading-rate hardware) that our scene and hardware did not have.

VII. Conclusion and Future Work

We implemented and measured a PI-controlled, bounded adaptive LOD for four vegetation categories in Godot. Against drawing everything, LOD raises average FPS from 21.4 to 96.8 at 1080p and from 19.6 to 72.1 at 1440p. The controller itself contributes +0.2% and +2.4% over fixed distances, engages only when the GPU exceeds its target, and reduces policy CPU time by roughly a factor of ten. Future work includes tuning floors, caps and gains; per-instance importance; cross-fading between tree meshes; a CPU-bound timing signal; release-build and multi-GPU measurements across power states; perceptual metrics such as SSIM or FLIP; and a systematic comparison with the literature. On the engine side, GPU-driven culling and visibility-buffer pipelines [41], [42], hardware instancing [43] and geometry clipmaps for terrain [44] are natural extensions.

Developer surveys of simplification [45] list further options that a feedback controller like ours could drive.

References

[1] J. H. Clark, “Hierarchical geometric models for visible surface algorithms,” Commun. ACM, vol. 19, no. 10, pp. 547–554, 1976.

[2] J. Rossignac and P. Borrel, “Multi-resolution 3D approximations for rendering complex scenes,” in Modeling in Computer Graphics. Berlin, Germany: Springer, 1993, pp. 455–465.

[3] D. Luebke, M. Reddy, J. D. Cohen, A. Varshney, B. Watson, and R. Huebner, Level of Detail for 3D Graphics. San Francisco, CA, USA: Morgan Kaufmann, 2003.

[4] T. Akenine-Möller, E. Haines, and N. Hoffman, Real-Time Rendering, 4th ed. Boca Raton, FL, USA: CRC Press, 2018.

[5] H. Hoppe, “Progressive meshes,” in Proc. ACM SIGGRAPH, 1996, pp. 99–108.

[6] J. Cohen et al., “Simplification envelopes,” in Proc. ACM SIGGRAPH, 1996, pp. 119–128.

[7] M. Garland and P. S. Heckbert, “Surface simplification using quadric error metrics,” in Proc. ACM SIGGRAPH, 1997, pp. 209–216.

[8] O. Deussen et al., “Realistic modeling and rendering of plant ecosystems,” in Proc. ACM SIGGRAPH, 1998, pp. 275–286.

[9] X. Décoret, F. Durand, F. X. Sillion, and J. Dorsey, “Billboard clouds for extreme model simplification,” ACM Trans. Graph., vol. 22, no. 3, 2003.

[10] P. Decaudin and F. Neyret, “Rendering forest scenes in real-time,” in Proc. Eurographics Symp. Rendering, 2004.

[11] K. Pelzer, “Rendering countless blades of waving grass,” in GPU Gems, R. Fernando, Ed. Boston, MA, USA: Addison-Wesley, 2004, ch. 7.

[12] E. Bruneton and F. Neyret, “Real-time realistic rendering and lighting of forests,” Comput. Graph. Forum, vol. 31, no. 2, 2012.

[13] P. Lindstrom et al., “Real-time, continuous level of detail rendering of height fields,” in Proc. ACM SIGGRAPH, 1996, pp. 109–118.

[14] B. Karis, R. Stubbe, and G. Wihlidal, “A deep dive into Nanite virtualized geometry,” in ACM SIGGRAPH Advances in Real-Time Rendering in Games course, 2021.

[15] T. A. Funkhouser and C. H. Séquin, “Adaptive display algorithm for interactive frame rates during visualization of complex virtual environments,” in Proc. ACM SIGGRAPH, 1993.

[16] M. Wimmer and P. Wonka, “Rendering time estimation for real-time rendering,” in Proc. Eurographics Workshop Rendering, 2003.

[17] Wong and Wang, “Interactive rendering of dynamic environment using PID control,” Eurographics short paper, 2006.

[18] K. J. Åström and T. Hägglund, PID Controllers: Theory, Design, and Tuning, 2nd ed. Research Triangle Park, NC, USA: ISA, 1995.

[19] “Adaptive LOD: progressive mesh control for stable real-time rendering in VR interior design,” The Visual Computer, 2026, doi: 10.1007/s00371-026-04784-5.

[20] “Real-time GPU tree generation,” in High-Performance Graphics, 2025, doi: 10.2312/hpg.20251168.

[21] “Real-time path tracing of massive dynamic foliage,” ACM, doi: 10.1145/3820021.

[22] Joubjang and Rakthanmanon, “Automatic level of detail for improved rendering by CPU usage,” in RSU Int. Research Conf., 2024.

[23] J. Dupuy et al., “Concurrent binary trees (with application to longest edge bisection),” Proc. ACM Comput. Graph. Interact. Tech., 2020, doi: 10.1145/3406186.

[24] Godot Engine, version 4.7.2-stable. [Online]. Available: https://godotengine.org

[25] Terrain3D, version 1.0.2, Godot terrain plugin.

[26] K. Ogata, Modern Control Engineering, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.

[27] K. J. Åström and R. M. Murray, Feedback Systems: An Introduction for Scientists and Engineers. Princeton, NJ, USA: Princeton Univ. Press, 2008.

[28] J. L. Hellerstein, Y. Diao, S. Parekh, and D. M. Tilbury, Feedback Control of Computing Systems. Hoboken, NJ, USA: Wiley, 2004.

[29] J. G. Ziegler and N. B. Nichols, “Optimum settings for automatic controllers,” Trans. ASME, vol. 64, pp. 759–768, 1942.

[30] J. Gregory, Game Engine Architecture, 3rd ed. Boca Raton, FL, USA: CRC Press, 2018.

[31] R. Jain, The Art of Computer Systems Performance Analysis. New York, NY, USA: Wiley, 1991.

[32] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in Proc. ACM OOPSLA, 2007, pp. 57–76.

[33] T. Kalibera and R. Jones, “Rigorous benchmarking in reasonable time,” in Proc. ACM ISMM, 2013, pp. 63–74.

[34] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, 2004.

[35] P. Andersson et al., “FLIP: a difference evaluator for alternating images,” Proc. ACM Comput. Graph. Interact. Tech., vol. 3, no. 2, 2020.

[36] R. Mantiuk, K. J. Kim, A. G. Rempel, and W. Heidrich, “HDR-VDP-2: a calibrated visual metric for visibility and quality predictions in all luminance conditions,” ACM Trans. Graph., vol. 30, no. 4, 2011.

[37] M. Reddy, “Perceptually modulated level of detail for virtual environments,” Ph.D. dissertation, Univ. Edinburgh, Edinburgh, U.K., 1997.

[38] B. Guenter, M. Finch, S. Drucker, D. Tan, and J. Snyder, “Foveated 3D graphics,” ACM Trans. Graph., vol. 31, no. 6, 2012.

[39] A. Patney et al., “Towards foveated rendering for gaze-tracked virtual reality,” ACM Trans. Graph., vol. 35, no. 6, 2016.

[40] K. Vaidyanathan et al., “Coarse pixel shading,” in Proc. High Performance Graphics, 2014, pp. 9–18.

[41] U. Haar and S. Aaltonen, “GPU-driven rendering pipelines,” in ACM SIGGRAPH Advances in Real-Time Rendering in Games course, 2015.

[42] C. A. Burns and W. A. Hunt, “The visibility buffer: a cache-friendly approach to deferred shading,” J. Comput. Graph. Techn., vol. 2, no. 2, pp. 55–69, 2013.

[43] F. Carucci, “Inside geometry instancing,” in GPU Gems 2, M. Pharr, Ed. Boston, MA, USA: Addison-Wesley, 2005, ch. 3.

[44] F. Losasso and H. Hoppe, “Geometry clipmaps: terrain rendering using nested regular grids,” ACM Trans. Graph., vol. 23, no. 3, pp. 769–776, 2004.

[45] D. Luebke, “A developer’s survey of polygonal simplification algorithms,” IEEE Comput. Graph. Appl., vol. 21, no. 3, pp. 24–35, 2001.